

Airlines Dataset — Data Pipeline Documentation

Submitted by Shamatmika Raja, CHRIST (Deemed to be University)
Date: September 2026

Table of Contents

1. Project Overview
2. Architecture Diagram
3. Data Flow Diagram
4. Data Model
5. Dataset Structure
6. Assumptions
7. Data Cleaning Logic and Strategy
8. Transformation Steps and Logic
9. Code Implementation Notes
10. Data Quality Governance
11. KPI Definitions

1. Project Overview

This document describes the end-to-end data engineering solution built for the Airlines Use-Case. The solution ingests raw flight scheduling and booking data from an Excel workbook and CSV files, applies a series of validated cleaning and transformation rules, and produces a cleaned, aggregated dataset ready for Power BI reporting.

The pipeline is implemented in Python using pandas and numpy, making it portable and reproducible across any standard data engineering environment, including Azure Databricks, AWS Glue, or a local notebook environment.

Key Objectives

- Ingest raw flight and booking data with full error handling and logging.
- Apply business-rule-driven data quality checks across all source datasets.
- Detect and correct overnight flights where arrival timestamps precede departure due to date roll-over.
- Protect passenger personally identifiable information through masking and field-level restriction.
- Produce a star-schema-ready cleaned dataset for Power BI consumption.
- Generate aggregated KPI tables to support dashboard tile and chart visuals.

2. Architecture Diagram

The solution follows a Medallion Architecture pattern with three logical layers: Raw (Bronze), Cleaned (Silver), and Aggregated (Gold).

Layer	Description	Output
Bronze (Raw)	Raw ingestion from Excel workbook and CSV files. No transformations applied.	raw_flights, raw_bookings, raw_passengers
Silver (Cleaned)	All cleaning, validation, and transformation rules applied. PII masked.	dim_flights, dim_bookings, dim_passengers_masked, fact_flight_operations
Gold (Aggregated)	KPI aggregations computed from the Silver fact table.	kpi_overview, kpi_by_airline, kpi_by_route, data_quality_log

Technology Stack: Python 3.12, pandas, numpy, openpyxl. The pipeline is designed to be lifted into Azure Data Factory, Databricks, or AWS Glue with minimal changes to the core transformation logic.

3. Data Flow Diagram

The following describes the data flow through the pipeline.

Step	Source	Process	Output
1	UseCase_-_Airlines.xlsx (sheet: flights)	Ingest Excel, parse datetimes, convert duration to minutes	raw_flights DataFrame
2	dim_bookings.csv	Ingest CSV, parse booking_date	raw_bookings DataFrame
3	dim_passengers_masked.csv	Ingest CSV, restrict to PII-safe columns	raw_passengers DataFrame
4	raw_flights	Apply 10 DQ rules: dedup, validate IDs, timestamps, overnight fix, duration recalc	dim_flights (VALID records)
5	raw_bookings	Standardise status, flag unknowns, derive is_confirmed	dim_bookings (cleaned)
6	raw_passengers	Column restriction to 4 PII-safe fields	dim_passengers_masked
7	dim_bookings + dim_flights	LEFT JOIN on flight_id, tag unmatched, compute payment amounts	fact_flight_operations
8	fact_flight_operations	GROUP BY airline, route; compute KPI metrics	kpi_overview, kpi_by_airline, kpi_by_route
9	All outputs	Write to CSV for Power BI import	8 cleaned CSV files

4. Data Model

The cleaned dataset follows a Star Schema with one central fact table and three dimension tables. This design is optimised for Power BI DirectQuery and Import Mode.

Fact Table

Table	Grain	Primary Key	Foreign Keys
fact_flight_operations	One row per booking	booking_id	flight_id, passenger_id

Dimension Tables

Table	Description	Primary Key	Cardinality
dim_flights	One row per unique flight schedule	flight_id	1,005 rows
dim_bookings	One row per booking	booking_id	1,000 rows
dim_passengers_masked	One row per passenger (PII-masked)	passenger_id	1,039 rows

Relationships

- fact_flight_operations.flight_id → dim_flights.flight_id (Many-to-One)
- fact_flight_operations.passenger_id → dim_passengers_masked.passenger_id (Many-to-One)
- fact_flight_operations.booking_id → dim_bookings.booking_id (One-to-One)

5. Dataset Structure

5a. Raw Source: UseCase_-_Airlines.xlsx (sheet: flights)

Column	Source Type	Description
flight_id	String	Unique alphanumeric flight identifier (e.g. SJ010, AI155)
airline	String	Carrier name. Blank cells mapped to Unknown.
source	String	IATA departure airport code (CCU, BOM, MAA, DEL, BLR, HYD)
destination	String	IATA arrival airport code
departure_time	datetime64	Scheduled departure datetime, parsed by openpyxl
arrival_time	datetime64	Scheduled arrival datetime, may cross midnight
duration	datetime.time	Flight duration as HH:MM:SS time object. Converted to minutes.

5b. dim_bookings.csv

Column	Type	Description
booking_id	String	Primary key (e.g. B1000)
passenger_id	String	Foreign key to dim_passengers_masked
flight_id	String	Foreign key to dim_flights
booking_date	datetime64	Timestamp of booking creation
status	String	CONFIRMED CANCELLED PENDING UNKNOWN

5c. dim_passengers_masked.csv

Column	Type	Description
passenger_id	String	Internal join key (e.g. P1000)

Column	Type	Description
passenger_key	String	SHA-256 hash of passenger name. No PII retained.
age_band	String	Age group: 0-17, 18-34, 35-54, 55+
gender	String	M or F

6. Assumptions

- All flight data originates from a single day of operations (20 April 2026). Departure and arrival timestamps share the same date for non-overnight flights, and differ by one calendar day for overnight flights.
- The IATA airport codes used are: BOM (Mumbai), DEL (Delhi), MAA (Chennai), CCU (Kolkata), BLR (Bengaluru), HYD (Hyderabad). These are treated as valid and no external code lookup is performed.
- A maximum domestic flight duration of 720 minutes (12 hours) is assumed. Any computed duration exceeding this value is rejected as an error.
- Flight identifiers are expected to follow the pattern: 2-letter or digit airline prefix followed by 2-4 digits (e.g. AI155, 6F026, SJ010). This is validated against a regex.
- Blank or null airline values in the source are labelled Unknown and retained rather than rejected, to preserve the maximum number of bookable records for revenue analysis.
- Payment amounts are not present in the raw source files. They are simulated using a reproducible random seed (seed=42) drawn uniformly from INR 3,000 to INR 25,000 for CONFIRMED and PENDING bookings. CANCELLED and UNKNOWN bookings carry zero payment.
- Passenger PII (name, contact, national ID) does not appear in the source. The passenger_key field is already hashed. No additional masking is required beyond column-level restriction.
- Duplicate flight rows with the same flight_id, departure time, source, and destination are considered scheduling conflicts. Only the first occurrence is retained.

7. Data Cleaning Logic and Strategy

Each cleaning rule is recorded in a data quality log (data_quality_log.csv) with the rule name and the number of rows affected. This log is included in the cleaned dataset export for audit purposes.

Rule	Logic	Action	Rows Affected
Missing airline	airline IS NULL or blank string	Fill with 'Unknown'	41
Exact duplicates	All columns identical	Drop all but first occurrence	15
Malformed flight ID	Does not match [A-Z0-9]{3,7}	Reject row	0
Same origin/destination	source == destination	Reject row	0
Missing timestamps	departure_time or arrival_time IS NULL	Reject row	0
Overnight flight correction	arrival_time < departure_time	Add +1 day to arrival_time	1
Invalid duration	duration_minutes <= 0 or > 720	Reject row	0
Conflicting schedule	Duplicate flight_id + departure_time + route	Keep first, reject rest	0
Invalid booking status	Status not in {CONFIRMED, CANCELLED, PENDING}	Relabel as UNKNOWN	75
Unmatched bookings	booking.flight_id not found in dim_flights	Tag REJECTED, retain for audit	0

8. Transformation Steps and Logic

8a. Duration Calculation

The raw Excel source stores the flight duration column as a datetime.time object in HH:MM:SS format. This is converted to fractional minutes using the formula:

$$\text{duration_minutes} = (\text{hours} * 60) + \text{minutes} + (\text{seconds} / 60)$$

After overnight flight correction (see Section 8b), duration_minutes is also independently recalculated directly from the corrected timestamps:

$$\text{duration_minutes} = (\text{arrival_time} - \text{departure_time}).\text{total_seconds}() / 60$$

The timestamp-derived value is used as the authoritative field in the fact table, as it accounts for any date corrections applied during the overnight flight step.

8b. Overnight Flight Handling

A flight is classified as overnight when its arrival_time timestamp is earlier than its departure_time timestamp — indicating the aircraft crossed midnight and the arrival date was not incremented in the source data.

Detection condition:

$$\text{arrival_time} < \text{departure_time}$$

Correction applied:

$$\text{arrival_time} = \text{arrival_time} + \text{timedelta(days=1)}$$

One overnight flight was detected and corrected in this dataset. The overnight_flight boolean column is set to True for these rows in both dim_flights and fact_flight_operations, enabling overnight segmentation in the dashboard.

8c. Route Derivation

The route column is a derived field constructed by concatenating source and destination:

$$\text{route} = \text{source} + '-' + \text{destination}$$

Examples: BOM-CCU, DEL-MAA, CCU-BLR. This string key enables route-level grouping in KPI aggregations.

8d. Booking Confirmation Flag

The is_confirmed boolean column is derived from the booking status field. It is True when status == CONFIRMED and False otherwise. This simplifies DAX measures and chart filters in the Power BI report.

8e. Fact Table Join Strategy

Bookings are joined to flights using a LEFT JOIN on flight_id. This preserves all bookings even when no matching flight record exists. Unmatched bookings receive flight_match_status = REJECTED and all flight-derived fields are filled with UNKNOWN or zero, preventing null propagation into KPI calculations.

9. Code Implementation Notes

- The pipeline is implemented in `airlines_pipeline.py` and the equivalent `airlines_pipeline.ipynb` Jupyter notebook.
- All logging is dual-channel: messages are written to a timestamped log file under `logs/` and echoed to `stdout`. This supports both notebook execution and headless pipeline runs.
- Error handling uses `try/except` blocks around all file I/O operations. A failure during ingestion raises an exception immediately; transformation errors are recorded in the quality log without halting the pipeline.
- The random seed for payment simulation is fixed at 42. This ensures all pipeline runs produce identical payment amounts, which is important for dashboard reproducibility.
- No external data is fetched. The pipeline is fully self-contained and offline-capable.
- All outputs are UTF-8 encoded CSVs without a row index, ensuring clean import into Power BI without spurious index columns.

10. Data Quality Governance

PII Protection

The passenger dataset is loaded with column-level restriction to four fields. No name, email, phone, or national identifier is loaded into memory at any stage. The passenger_key field is a SHA-256 hash of the original name and is retained only as a deduplication key.

Audit Trail

Every data quality rule applied during the pipeline run is recorded in data_quality_log.csv. This file contains the rule name and the number of rows affected, and is included in the cleaned data export. It is also loaded into Power BI to enable dashboard-level data quality reporting.

Idempotency

The pipeline is idempotent: running it multiple times with the same source files produces identical output files. The random seed for payment amounts is fixed, and all deduplication rules are deterministic (first-occurrence wins).

11. KPI Definitions

KPI	Formula	Source Table	Notes
Total Bookings	COUNT(booking_id)	fact_flight_operations	All bookings including CANCELLED
Confirmed Bookings	SUM(is_confirmed)	fact_flight_operations	status = CONFIRMED only
Confirmation Rate	Confirmed / Total * 100	fact_flight_operations	Percentage
Valid Flight Records	COUNT WHERE flight_match_status = VALID	fact_flight_operations	Excludes unmatched bookings
Average Duration (min)	MEAN(duration_minutes) WHERE duration > 0	fact_flight_operations	Excludes unmatched rows
Overnight Flights	SUM(overnight_flight)	fact_flight_operations	Boolean sum
Total Payment Revenue	SUM(payment_amount)	fact_flight_operations	CONFIRMED + PENDING only
Revenue per Airline	SUM(payment_amount) GROUP BY airline	kpi_by_airline	Sorted descending
Top Route by Bookings	MAX(booking_count) from kpi_by_route	kpi_by_route	Single value